

TryHackMe — Computer Fundamentals Module

Virtualisation Basics

Learning Notes — Hypervisors, VMs, Containers, Docker & VM Management

"Have you ever considered how expensive and inefficient it would be if every piece of software or every website required its own physical server? Virtualisation was created to solve exactly this problem."



1. What Is Virtualisation?

Virtualisation is the technology that enables a single physical computer to act like multiple separate, independent computers simultaneously. Instead of dedicating one physical server to each application or operating system, virtualisation allows many virtual environments to share the same underlying hardware — dramatically improving efficiency, flexibility, and cost-effectiveness.

The Core Problem Virtualisation Solves

Have you ever considered how expensive and inefficient it would be if every piece of software or every website required its own dedicated physical server? Virtualisation was created to solve exactly this problem. A single powerful server can now run dozens of virtual machines, each with its own OS, isolated from the others — replacing what would have been dozens of separate physical machines.

1.1 The Key Benefits of Virtualisation

Benefit	What It Means in Practice
Cost Efficiency	Fewer physical machines needed — one powerful server replaces many. Lower hardware purchase, maintenance, power, and cooling costs.
Scalability	New virtual machines can be spun up in minutes without purchasing new hardware. Scale up by adding more VMs; scale down by deleting them.
Flexibility	Different operating systems (Windows, Linux, macOS) can run simultaneously on the same physical hardware — no need for separate machines per OS.
Isolation	Each VM is completely isolated. A crash, virus, or misconfiguration in one VM does not affect other VMs on the same host.
Resource Utilisation	Physical hardware is often underutilised — virtualisation allows multiple workloads to share resources efficiently, maximising hardware ROI.
Snapshots & Recovery	VM state can be saved as a snapshot at any moment — instantly revert to a known-good state after a crash or failed update.
Portability	VMs can be exported as files and migrated to different physical hosts — enabling disaster recovery, cloud migration, and load balancing.
Safe Testing	Create isolated lab environments for security research, software testing, and certification study without risking production systems.

Room Answer: What does virtualisation enable multiple applications to share?

Physical server — multiple VMs share the same underlying physical server hardware, managed by the hypervisor. Room Answer: What is the name of the software that manages the resources for each VM? Hypervisor.

2. The Hypervisor — The Manager of Virtual Machines

A hypervisor (also called a Virtual Machine Monitor, or VMM) is the software layer that creates, runs, and manages virtual machines. It acts as an intermediary between the physical hardware and the virtual machines — allocating CPU time, RAM, storage, and network resources to each VM and ensuring they remain isolated from each other.

2.1 Type 1 Hypervisor — Bare Metal

Type 1 Hypervisor — Installed directly on physical hardware — runs without an underlying OS

- Runs directly on the physical host hardware — there is no underlying operating system between the hypervisor and the hardware.
- Also called a 'bare metal hypervisor' — it IS the operating system layer for the physical machine.
- Designed for enterprise data centres, server farms, and cloud infrastructure.
- Requires a separate management console for administration — not a typical user interface.
- Maximum performance — direct hardware access, minimal overhead, no host OS consuming resources.
- Maximum isolation and security — no host OS vulnerabilities to exploit beneath the hypervisor.
- Examples: VMware ESXi, Microsoft Hyper-V (server mode), Citrix XenServer, Oracle VM Server, KVM (Linux kernel-based).
- Room answer: A company wants to host servers in a data centre → Type 1 hypervisor.

Security note: Type 1 hypervisors are the foundation of cloud infrastructure (AWS, Azure, GCP all use bare-metal hypervisors). Compromising a Type 1 hypervisor gives an attacker control over ALL VMs running on that host — a 'hypervisor escape' is one of the most critical cloud security vulnerabilities.

2.2 Type 2 Hypervisor — Hosted

Type 2 Hypervisor — Installed as an application on an existing host OS — runs on top of Windows, Linux, or macOS

- Runs as a standard application on top of an existing host operating system (Windows, Linux, macOS).
- Also called a 'hosted hypervisor' — the host OS manages hardware; the hypervisor sits above it.
- More overhead than Type 1 — host OS consumes resources before the hypervisor even starts.
- Easier to install and use — no dedicated hardware or management console needed.
- Ideal for personal use, development, testing, security labs, and certification study environments.
- VMs run inside windows on the host desktop — easy to switch between host and VMs.
- Examples: VMware Workstation, Oracle VirtualBox, Parallels Desktop (macOS), VMware Fusion (macOS).
- Room answer: A user deploying a study lab for a cyber security certification → Type 2 hypervisor.

Security note: Type 2 hypervisors inherit the security vulnerabilities of the host OS — a compromised host OS can affect all VMs. VMs also inherit network exposure from the host's network interface. Malware in a VM can potentially escape to the host through Type 2 hypervisor vulnerabilities (VM escape).

2.3 Type 1 vs Type 2 — Full Comparison

Feature	Type 1 — Bare Metal	Type 2 — Hosted
Runs on	Directly on physical hardware — no host OS	On top of a host OS (Windows, Linux, macOS)
Also called	Bare metal hypervisor	Hosted hypervisor
Performance	Maximum — direct hardware access, minimal overhead	Lower — host OS consumes resources first
Ease of setup	Complex — requires dedicated hardware, mgmt console	Simple — installs like any application
Best for	Enterprise data centres, cloud providers, servers	Personal labs, development, certification study, testing
Management	Separate management console (vCenter, XenCenter)	Normal desktop UI — run VMs in windows
Resource isolation	Near-complete — hardware dedicated to hypervisor	Shares resources with host OS processes
Examples	VMware ESXi, Hyper-V, XenServer, KVM	VMware Workstation, VirtualBox, Parallels
Room scenario	Company hosting servers in data centre	User deploying a cyber security study lab

3. Virtual Machines (VMs)



Virtual Machine (VM) — A whole virtual computer inside the real one — with its own OS, CPU, RAM, and storage

- A VM is a complete software emulation of a physical computer — it has virtual CPU cores, virtual RAM, virtual storage, and a virtual network interface.
- Each VM runs its own independent operating system — completely isolated from other VMs on the same host.
- The hypervisor allocates physical resources (CPU time, RAM, disk, network bandwidth) to each VM according to its configuration.
- From the perspective of the OS running inside the VM, it appears to be running on real physical hardware — it cannot tell it is virtualised.
- VMs can run any compatible OS — Linux, Windows, BSD, macOS — regardless of the host OS.
- Snapshots: the entire state of a VM (RAM, disk, CPU state) can be saved as a point-in-time snapshot and restored instantly.
- Cloning: VMs can be duplicated — useful for creating multiple identical lab environments or scaling out web servers.
- Live migration: running VMs can be moved between physical hosts with minimal downtime (used in enterprise environments).

Security note: VMs provide strong isolation — malware in a VM cannot directly access the host system (unless a VM escape vulnerability exists). This makes VMs ideal for malware analysis sandboxes. Each VM has its own network stack, making it easy to set up isolated lab networks for security testing.

3.1 VM Resource Allocation

Resource	How It Is Virtualised
CPU	Physical CPU cores are shared across VMs through time-slicing. Each VM is allocated virtual CPUs (vCPUs). Modern CPUs have hardware virtualisation extensions (Intel VT-x, AMD-V) for efficiency.
RAM	Physical RAM is partitioned and allocated to each VM. Memory ballooning and swapping allow the hypervisor to dynamically manage RAM across VMs.
Storage	VM disks are stored as image files (.vmdk, .vhd, .qcow2) on the host's physical storage. Each VM sees its own virtual drive — isolated from other VMs.
Network	Virtual NICs connect VMs to virtual switches (vSwitches). VMs can be on the same isolated virtual network or connected to the external physical network.
GPU	Modern hypervisors support GPU passthrough or virtualisation (vGPU) — sharing graphics processing capability across VMs.

4. Containers

Containers are a lightweight alternative to VMs for isolating and deploying applications. Rather than virtualising an entire computer with its own OS, containers share the host OS kernel and isolate only the application and its dependencies. This makes containers much faster to start, more resource-efficient, and highly portable.

 **Container** — A lightweight isolated box for one application — shares the host OS kernel

- A container packages an application and all its dependencies (libraries, config files, runtime) into a single portable unit.
- Containers share the host operating system's kernel — unlike VMs, they do NOT run a separate OS instance.
- Because there is no OS to boot, containers start in milliseconds (vs minutes for VMs).
- Containers are isolated from each other — a misbehaving container does not affect others running on the same host.
- Containers run consistently on any machine with a compatible container runtime — solving the 'works on my machine' problem.
- Ideal for: microservices architecture, CI/CD pipelines, scalable web deployments, development environments.
- Container images are immutable blueprints for creating containers — like a pre-packed recipe/template.
- Room answer: A company wants to host multiple small applications in the same VM → Containers.

Security note: Containers share the host OS kernel — a kernel vulnerability exploited from within a container can potentially affect the host or all containers. Container escape attacks target this shared kernel. However, namespaces and cgroups provide strong isolation for most use cases. Always run containers with minimal privileges and as non-root users.

4.1 Container Images

A container image is an immutable, pre-packaged template used to create containers. Think of it as a recipe: it contains everything needed to run the application — OS libraries, application code, runtime, environment variables, and configuration. Every container created from the same image starts in an identical state.

- Images are built in layers — each instruction adds a layer on top of the previous one (efficient caching and storage).
- Images are stored in registries — Docker Hub is the public registry; organisations use private registries.
- An image can spawn many identical containers — scale out by creating more containers from the same image.
- Container images are version-controlled — tagged with version numbers (e.g. nginx:1.25, python:3.12).

The Restaurant Analogy

A container image is like a recipe card. A container is like the dish cooked from that recipe. You can cook many identical dishes from one recipe — just as you can run many identical containers from one image. The recipe does not change when you cook from it — the image remains immutable each time a container is created from it.

5. Docker — The Standard Container Platform



Docker — The open-source platform that makes building, deploying, and running containers simple

- Docker is an open-source software platform that simplifies building, deploying, and running containerised applications.
- The easiest and most widely used way to deploy containers — both on personal machines and in production.
- Docker containers can be deployed inside a VM — a common pattern: Type 2 hypervisor runs a VM, containers run inside the VM.
- Docker uses Linux kernel features (namespaces and cgroups) to provide isolation and resource limits.
- Dockerfile: a text file with instructions for building a Docker image (FROM, RUN, COPY, EXPOSE, CMD).
- Docker Compose: a tool for defining and running multi-container applications in a single YAML file.

- Docker Hub: the public registry containing thousands of pre-built images (nginx, ubuntu, postgres, redis).
- Docker Desktop: the GUI application for running Docker on Windows and macOS.

Security note: Docker images from public registries (Docker Hub) may contain vulnerabilities or even malicious code — always use trusted, verified images. Run containers with the least privilege required. Exposed Docker socket (docker.sock) gives root-equivalent access to the host — never expose it unnecessarily.

5.1 Key Docker Commands

```
# Pull an image from Docker Hub
docker pull nginx:latest

# Run a container from an image
docker run -d -p 8080:80 nginx
# -d: run in background (detached)
# -p 8080:80: map host port 8080 to container port 80

# List running containers
docker ps

# Stop a running container
docker stop <container_id>

# List all images
docker images

# Remove a container
docker rm <container_id>
```

5.2 Network Ports in Containers

Network ports are the numbered entry points that applications use to communicate over a network. When a containerised application needs to be reachable from outside, its internal port must be mapped to a port on the host machine using the `-p` flag.

Port Concept	Explanation
Container port	The port the application inside the container listens on (e.g. a web server listens on port 80).
Host port	The port on the physical host machine that external traffic connects to (e.g. port 8080).
Port mapping (-p)	Maps host port to container port: <code>-p 8080:80</code> means traffic on host:8080 is forwarded to container:80.

Port 80	Default HTTP port — a web server container typically listens on this internally.
Port 443	Default HTTPS port — for TLS-encrypted web traffic inside the container.
Isolation benefit	Containers on the same host can each listen on port 80 internally — their host ports are mapped to unique values.

6. VMs vs Containers — The Apartment Analogy

The room uses a powerful analogy to distinguish VMs from containers: VMs are full apartments; containers are rooms within an apartment. Both provide isolation, but at very different levels:

Feature	Virtual Machine	Container
Analogy	A full apartment — self-contained, own utilities, own structure	A room in an apartment — private space but shares building infrastructure
OS	Full separate OS per VM (Windows, Linux, BSD)	Shares the host OS kernel — no separate OS instance
Size	GB range — full OS + application data	MB range — only application + dependencies
Startup time	Minutes — must boot an entire OS	Milliseconds — no OS to boot
Isolation	Maximum — complete hardware virtualisation, separate kernel	Strong but shares kernel — container escape possible
Resource use	Higher — full OS consumes CPU, RAM, storage	Much lower — minimal overhead per container
Portability	Portable but heavier — image files are large (GBs)	Highly portable — images are small (MBs), pull in seconds
Best for	Full system isolation, different OSes, strong security boundary	Microservices, scalable apps, CI/CD, fast deployment
Security model	Each VM is fully isolated — VM escape is rare but catastrophic	Shared kernel — kernel vulnerabilities affect all containers

The Room's Summary

VMs provide the 'full apartment' with maximum separation and flexibility, while containers offer lightweight 'rooms' ideal for scalable, fast-deploying applications. The two technologies are complementary — containers often run inside VMs in production environments, combining VM-level isolation at the host boundary with container-level efficiency for application deployment.

7. The Architecture Stack — How Everything Fits Together

Understanding how physical hardware, hypervisors, VMs, and containers layer on top of each other is essential for both infrastructure design and security assessment. Here is the complete stack:

Layer	What Lives Here
Physical Hardware	CPU, RAM, storage, network cards — the actual physical machine (server). Shared by all layers above it.
Type 1 Hypervisor	Bare metal hypervisor (ESXi, Hyper-V, KVM) installed directly — manages hardware for all VMs above it.
Virtual Machines	Each VM gets a slice of hardware resources. Runs its own OS (Windows Server, Ubuntu, CentOS) inside.
Type 2 Hypervisor	If used, runs as an application inside a VM or on a host OS — manages containers/VMs above it.
Container Runtime	Docker, containerd, or Podman installed inside a VM — manages container lifecycle.
Containers	Individual isolated application environments — each running one microservice, web app, or database.
Applications	The actual software (web server, API, database) running inside containers, served to users.

8. Practical Lab — AutoGalo VM Management

After learning the theory, the room puts you in the role of the person responsible for virtualisation at AutoGalo — a company that has recently started using virtual machines. You work with a Virtualisation Manager dashboard to inspect and manage a fleet of VMs.

8.1 The Four Lab Tasks

#	Task	How to Find It	Answer
4.1	What is the name of the VM that has been running for the longest time?	Sort by Uptime column — descending	Monitoring-SYS
4.2	What is the name of the VM using the biggest amount of memory?	Sort by Memory column — descending	DB-Cluster-01
4.3	How many VMs are in the running state after fixing Mail-SERVER issue?	Click the blue square button to fix Mail-SERVER2 error → count	8

4.4	What is the name of the physical machine hosting most of the VMs?	Check the Host Monitoring section of the dashboard	(Check dashboard)
-----	---	--	-------------------

Lab Skill: VM State Management

The lab demonstrates a core VM management skill: identifying and fixing a VM in an error state. Mail-SERVER2 is in an Error state — clicking the blue square button changes its status to Running. This is the equivalent of a hard reset in a virtualised environment — resolving a hung or failed VM without touching physical hardware. After the fix, there are 8 VMs in the running state.

9. Room Answers — Quick Reference

Question	Answer
What does virtualisation enable multiple applications to share?	physical server
What is the name of the software that manages resources for each VM?	hypervisor
Is scalability a primary benefit of virtualisation? (Y/N)	Y
Suppose a user deploys a study lab for a cyber security cert — Type 1 or Type 2?	type 2
Suppose a company wants to host multiple small apps in the same VM — what to use?	containers
What is the name of the VM running for the longest time?	Monitoring-SYS
What is the name of the VM using the biggest amount of memory?	DB-Cluster-01
How many VMs are running after fixing the Mail-SERVER issue?	8

10. Security Implications

Hypervisor Compromise = Total Control

A hypervisor controls all VMs running on a physical host. Compromising the hypervisor gives an attacker unrestricted access to every VM — their data, their memory, their network traffic. This is why hypervisor security is treated as the most critical layer in virtualised infrastructure. Hypervisors must be patched immediately when vulnerabilities are disclosed.

VM Escape — Breaking Out of the Box

VM escape vulnerabilities allow malware inside a VM to break out and affect the host system or hypervisor. While rare, they represent the most critical virtualisation security risk. Known examples include CVE-2015-3456 (VENOM — a QEMU floppy controller vulnerability) and various VMware and VirtualBox CVEs. Keeping hypervisors patched and using VMs from trusted sources mitigates this risk.

Snapshots Are a Double-Edged Sword

VM snapshots are invaluable for security labs and recovery — but they are also a security risk. Snapshots preserve the state of a VM including decrypted data, credentials in memory, and running malware. An attacker with access to snapshot files can extract sensitive data. Snapshots should be encrypted, and old snapshots should be deleted when no longer needed.

Containers Share the Kernel — Shared Risk

Unlike VMs, containers share the host OS kernel. A kernel vulnerability exploited from inside a container affects the entire host and all containers on it. Container security requires: running containers as non-root, using read-only filesystems where possible, enabling seccomp and AppArmor profiles, and keeping the host kernel patched.

Docker Socket Exposure = Root Access

The Docker daemon socket (`/var/run/docker.sock`) provides full control over all Docker operations on the host. Mounting this socket inside a container gives that container root-equivalent access to the host system — a critical privilege escalation path found in many misconfigured environments. This should never be mounted except in explicitly trusted scenarios.

Virtualisation Powers Security Labs

From a defensive perspective, virtualisation is invaluable for security: isolated sandboxes for malware analysis, network labs for penetration testing practice (TryHackMe itself uses VMs), forensic investigation environments, and blue team simulation ranges. Security professionals use Type 2 hypervisors (VirtualBox, VMware Workstation) daily for their lab work.

11. Key Takeaways & Glossary

11.1 Key Takeaways

- Virtualisation enables one physical server to run many virtual machines — sharing hardware efficiently.
- The hypervisor manages resources for each VM — allocating CPU, RAM, storage, and network.
- Type 1 (bare metal): installed directly on hardware — enterprise data centres, cloud providers. No host OS.
- Type 2 (hosted): installed as an app on top of a host OS — personal labs, development, security study.
- Study lab for a cyber security cert → Type 2. Company hosting data centre servers → Type 1.
- VMs are full apartments — own OS, maximum isolation. Containers are rooms — share kernel, ultra-fast.
- Multiple small applications in the same VM → use Containers.
- Container images are immutable templates; containers are running instances created from images.
- Docker is the standard container platform — simplifies building, deploying, and running containers.
- Port mapping (-p hostPort:containerPort) exposes container services to the outside world.
- Scalability is a primary benefit of virtualisation — Y.
- Lab answers: Monitoring-SYS (longest uptime), DB-Cluster-01 (most memory), 8 (running VMs after fix).

11.2 Core Glossary

Term	Definition
Virtualisation	Technology enabling a single physical computer to act as multiple independent virtual computers.
Hypervisor	The software that creates, runs, and manages virtual machines — allocates physical resources to each VM.
VMM	Virtual Machine Monitor — another name for a hypervisor.
Type 1 Hypervisor	Bare metal hypervisor — runs directly on hardware without an underlying OS. Enterprise/cloud standard.
Type 2 Hypervisor	Hosted hypervisor — runs as an application on top of a host OS. Used for personal labs and development.
Virtual Machine	A complete software emulation of a physical computer — own OS, CPU, RAM, storage, network interface.
VM Escape	A vulnerability allowing malware inside a VM to break out and affect the hypervisor or host system.

Snapshot	A point-in-time save of a VM's complete state — enables instant rollback to a known-good configuration.
Live Migration	Moving a running VM between physical hosts with minimal downtime — used for load balancing and maintenance.
Container	A lightweight isolated environment for one application that shares the host OS kernel.
Container Image	An immutable template/blueprint for creating containers — includes app code, dependencies, and config.
Docker	The open-source container platform for building, deploying, and running containerised applications.
Docker Hub	The public registry of Docker images — thousands of pre-built official and community images.
Dockerfile	A text file of instructions for building a Docker image (FROM, RUN, COPY, EXPOSE, CMD).
Port Mapping	-p hostPort:containerPort — maps a container's internal port to a port on the host machine.
Namespace	Linux kernel feature used by containers to isolate process, network, and filesystem views.
cgroups	Linux control groups — limit and monitor resource usage (CPU, RAM) by container processes.
VM Image	A file containing the complete virtual hard disk of a VM (.vmdk, .vhd, .qcow2 formats).
vCPU	Virtual CPU — a CPU core allocation assigned to a VM by the hypervisor.
vSwitch	Virtual switch — connects VMs on the same host to each other and/or the physical network.

11.3 Next Steps on TryHackMe

- **Immediate:** Continue with the Cloud Computing Fundamentals room — virtualisation powers cloud infrastructure.
- **Foundation:** Study the Virtualization and Containers advanced room for deeper Type 1/Type 2 and container content.
- **Foundation:** Complete the Linux Fundamentals rooms — Linux is the most common OS in VMs and containers.
- **Applied:** Take the Intro to Docker room — hands-on container building, running, and exploitation.
- **Security:** Study the Cloud Security rooms — cloud platforms are built entirely on virtualisation at scale.

Virtualisation Powers Everything

Every cloud service (AWS, Azure, GCP), every cyber security lab (including TryHackMe), every CI/CD pipeline, and most enterprise IT infrastructure runs on virtualisation. Understanding hypervisors, VMs, and containers is not optional background knowledge — it is the infrastructure layer that every security professional encounters every day, whether defending it or assessing it.

Source: TryHackMe — Virtualisation Basics Room | tryhackme.com/room/virtualisationbasics